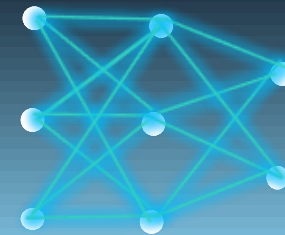
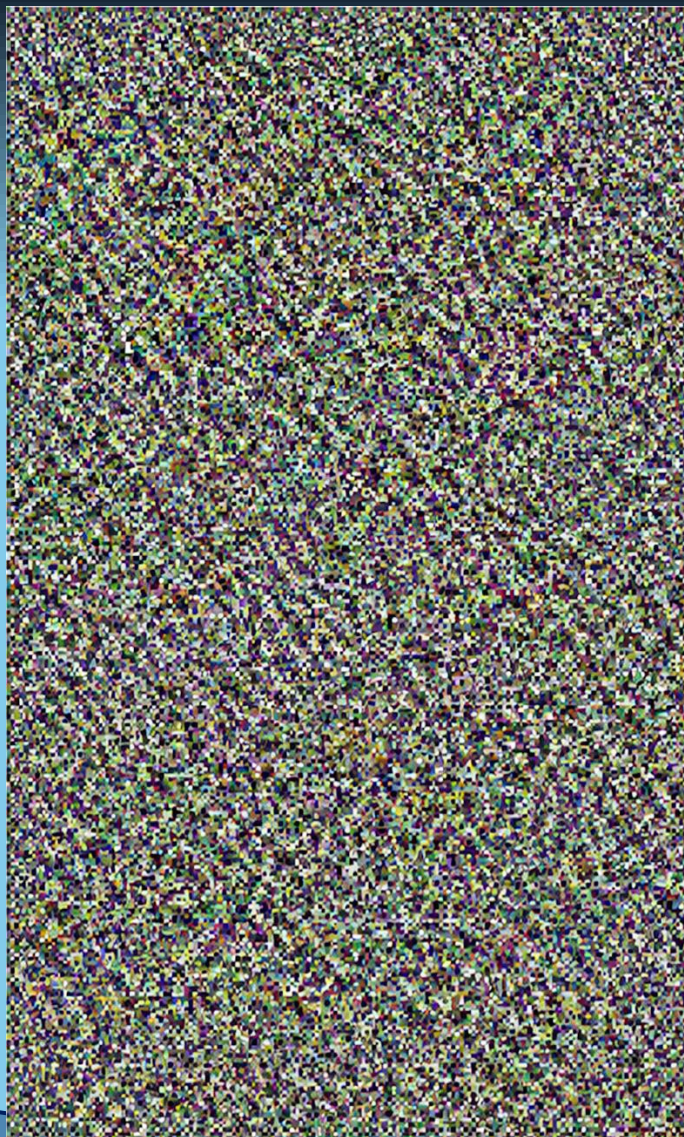
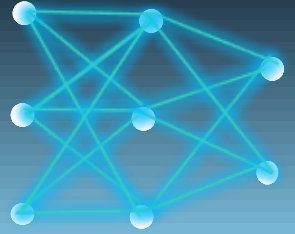
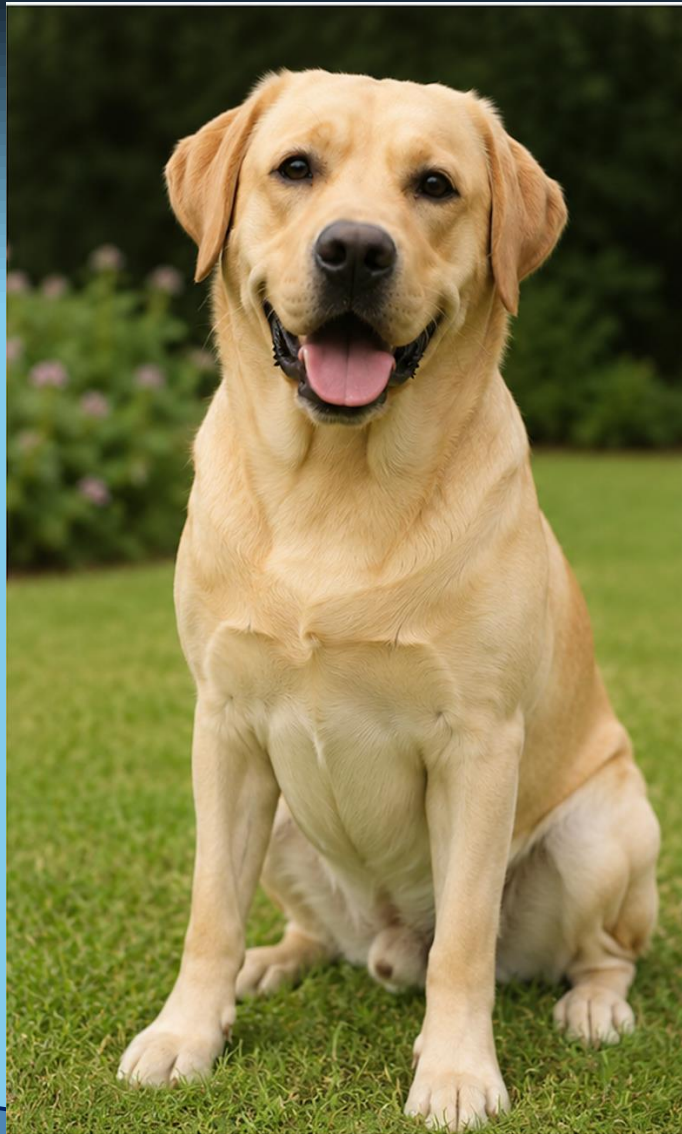


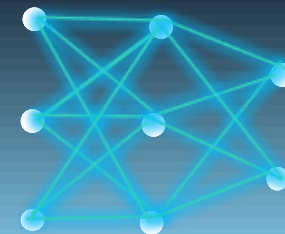


Deep Learning

Dr. Shay Maymon

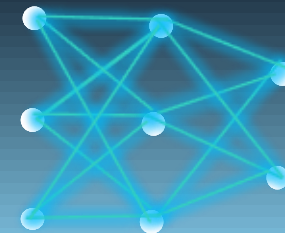






Unstructured Data Assumption

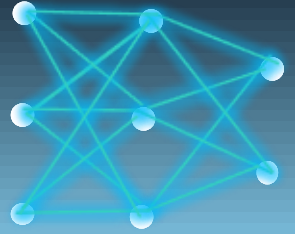
- Classical ML models assume **unstructured inputs**
- $\mathbf{x} = (x_1, \dots, x_d)$ treated as independent features
- No prior assumptions about relationships between dimensions
- **Key property:**
 - Any fixed permutation of features $\rightarrow$ **no change in model performance**
 - $\Rightarrow$ **Models are invariant to feature ordering**



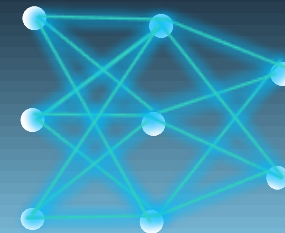
Limitation of This Assumption

- Assumes all input variables are **exchangeable**
- Ignores:
 - **locality**
 - **correlations**
 - **hierarchical structure**
- **Consequences:**
 - No spatial or temporal bias
 - Inefficient representation of real-world data

Structured Data in ML



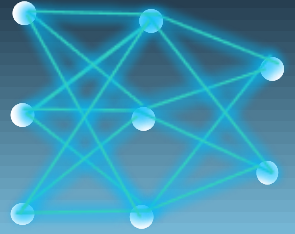
- Many real-world problems involve **structured inputs**:
 - **Natural Language**
 - **Images**



How Structure Is Exploited

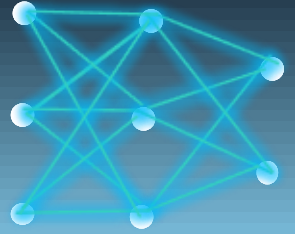
- Structure can be incorporated via:
 - Regularization terms
 - Data augmentation
 - Architectural design choices
- These methods enforce:
 - **Invariance** (robustness to transformations)
 - **Equivariance** (predictable transformation behavior)

Convolutional Neural Networks (CNNs)



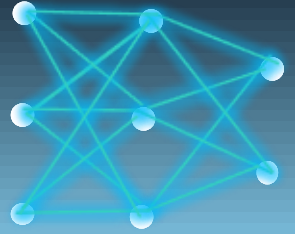
- CNNs are designed for structured data (especially images)
- **Key idea:**
 - Sparse connectivity
 - Parameter sharing across space
 - Designed to encode invariances and equivariances specific to image data

Computer Vision



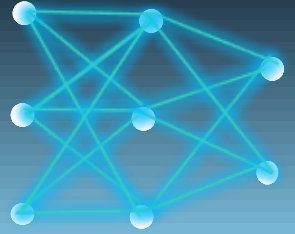
- **Computer vision** focuses on automatic analysis and interpretation of images
- It is a major application area for **machine learning**
- Early CV relied on:
 - 3D projective geometry
 - Hand-crafted features were used as inputs to simple learning algorithms

Deep Learning Revolution in Vision



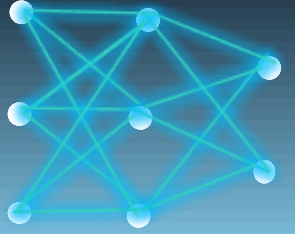
- Computer vision was among the first fields transformed by **deep learning, thanks to Convolutional Neural Networks (CNNs)**
- From hand-crafted features → learned hierarchical representations

Beyond CNNs



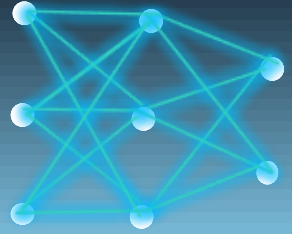
- CNNs were originally developed in the context of image analysis
- Later extended to:
 - Sequential data (e.g., language, time series)
- **Transformers** now compete with CNNs in vision tasks

Core Computer Vision Tasks



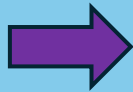
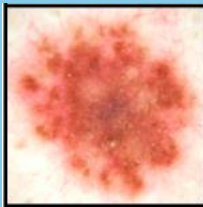
Machine learning in vision includes multiple problem types

Image Classification

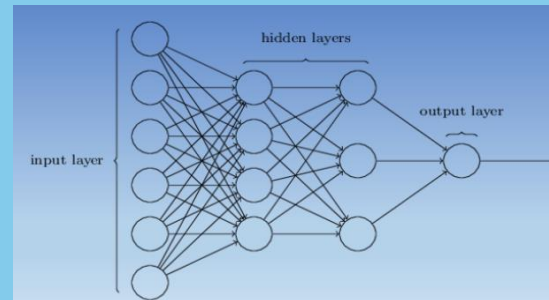


Assign a single label to an image

A new lesion



Trained Network

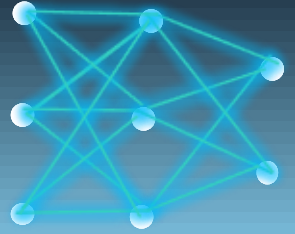


malignant melanomas

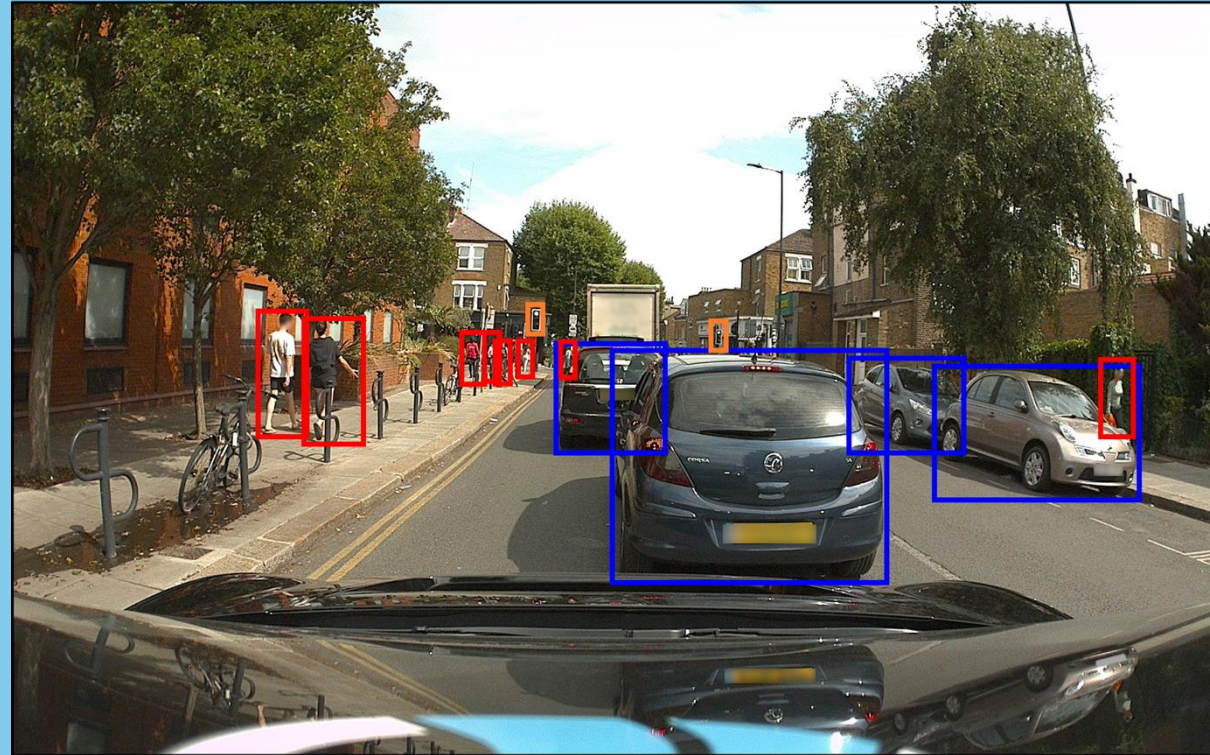
or

benign nevi

Object Detection



- Identify objects + their spatial locations
- Output:
 - class labels
 - bounding boxes



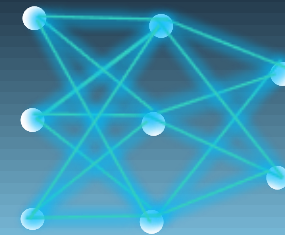
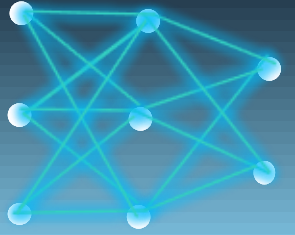
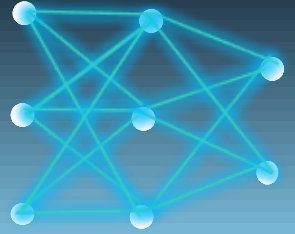


Image Segmentation



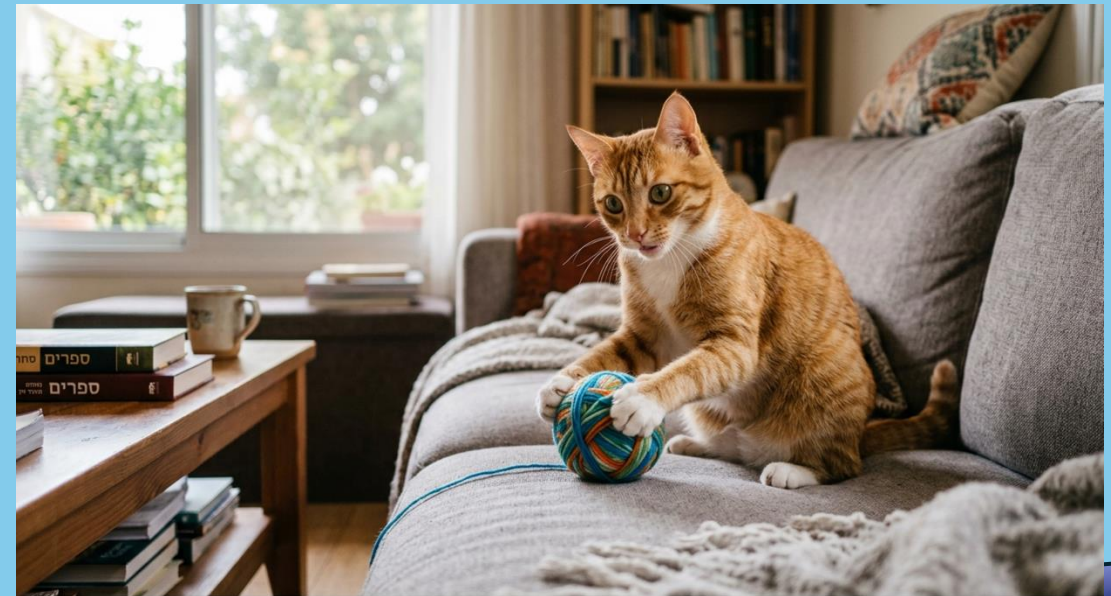
- Assign a label to **each pixel**
- Produces pixel-wise semantic regions

Caption Generation

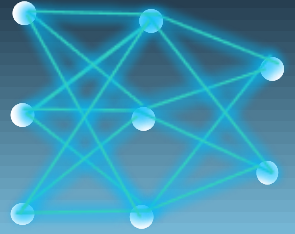


- Image $\rightarrow$ textual description

A cat playing with a colorful wool yarn ball on a cozy grey sofa in a sunlit living room



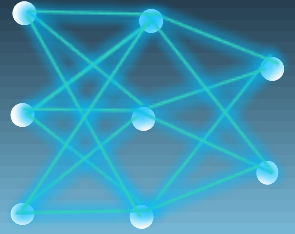
Synthesis



- Synthesis of new images



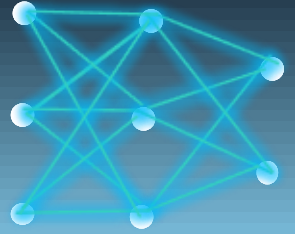
Inpainting



- Fill missing or removed regions consistently



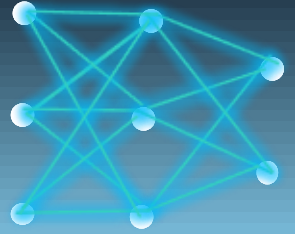
Style Transfer



- Transform image style (e.g., photo $\rightarrow$ painting)



Super-resolution



- Increase resolution + synthesizing high-frequency details

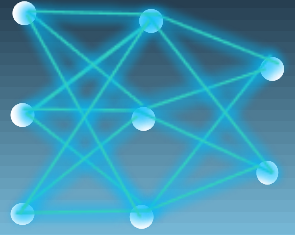


Input

Output

Original

Depth Prediction



- Estimate distance per pixel

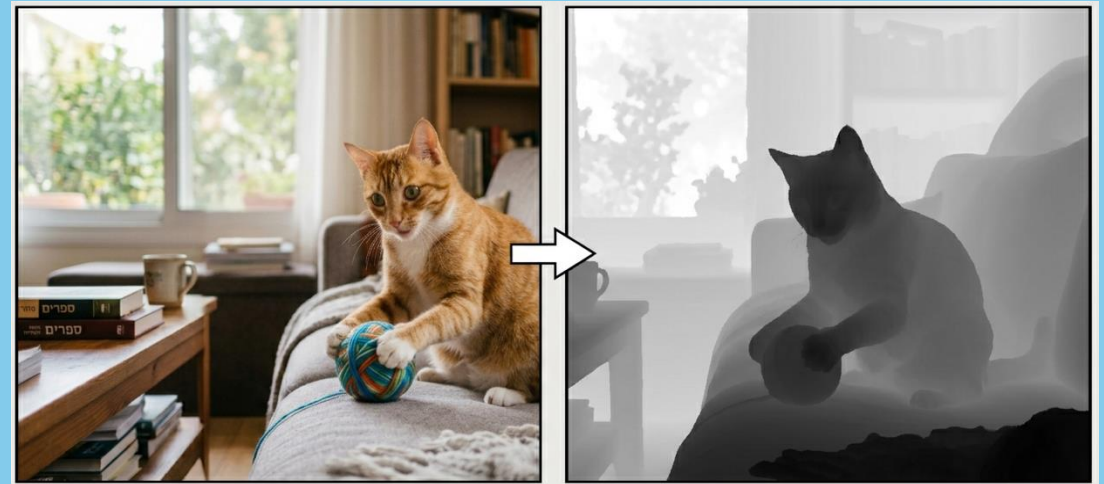
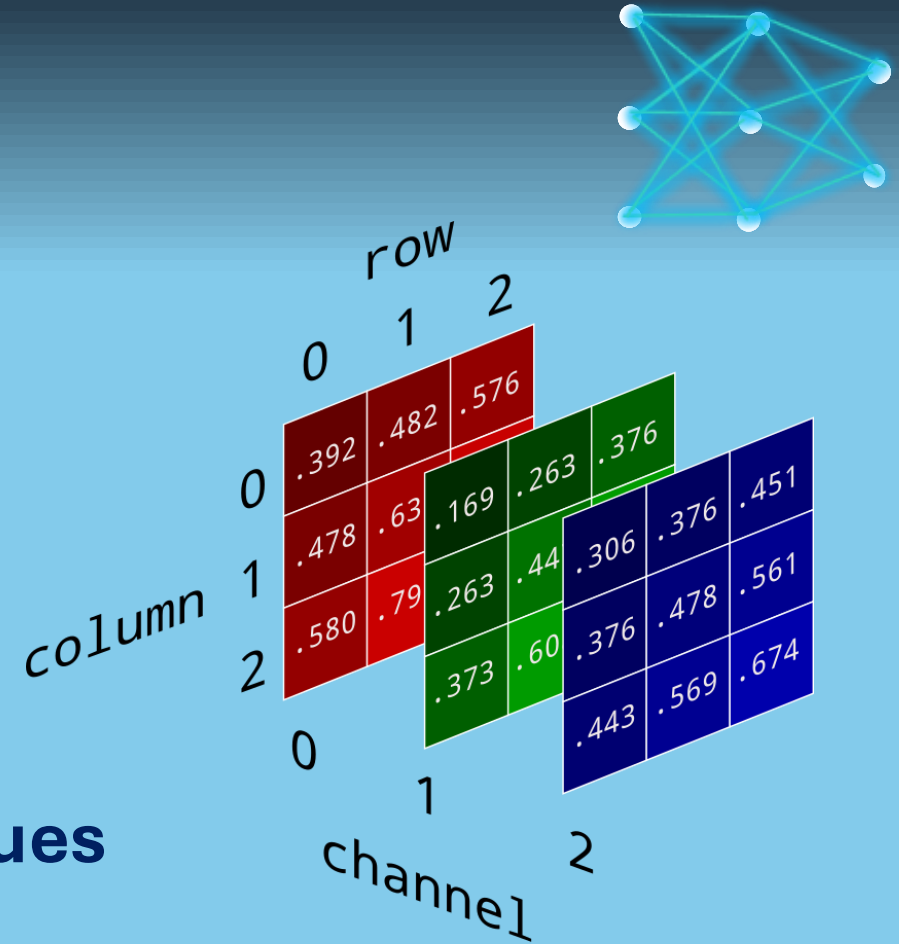
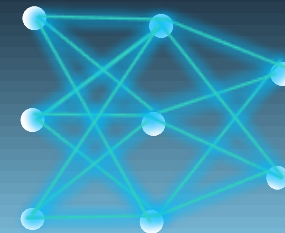


Image Data

- An image is a **rectangular array of pixels**
- Each pixel contains:
 - Grayscale intensity
or
 - **RGB** vector
- **Key properties:**
- Intensities are **non-negative bounded values**
- Limited by sensor / camera hardware

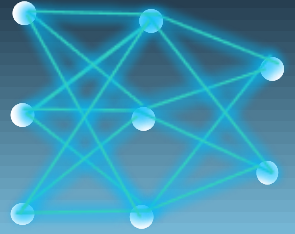




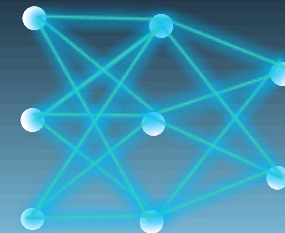
Data Representation in Practice

- Pixel values are often treated as **continuous variables**
- In practice, intensities are represented with finite precision
 - Commonly **8-bit integers**
 - Range: **0–255**

The Challenge of Applying Neural Networks



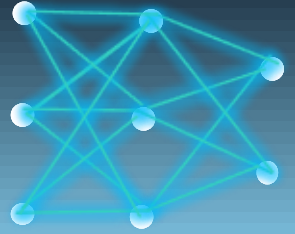
- Images are **high-dimensional inputs**
 - modern cameras: tens of megapixels
- **NAIVE approach:**
 - Treating the image as **unstructured vector** may require a model with an extremely large number of parameters - **Infeasible to train**
 - Flattening pixels destroys spatial structure
 - **Fails to take account of the highly structured nature of image data – local correlations are important**



Local Correlations in Images

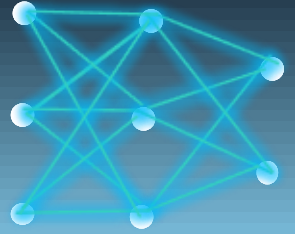
- Natural images exhibit strong **local dependency**
- Nearby pixels are highly correlated:
 - similar colors
 - similar intensities
- Far-away pixels:
 - weak correlation

Inductive Bias from Image Structure



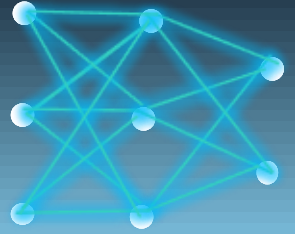
- Image structure provides strong **prior knowledge**
- This can be used to encode strong inductive biases into a neural network
- **Benefits:**
 - fewer parameters
 - better generalization accuracy

Motivation for Convolutional Networks



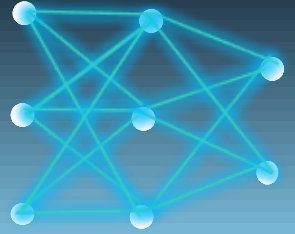
- Fully connected networks are **impractical for images**
- Example: A color image
 - Resolution: $10^3 \times 10^3$ pixels
 - RGB channels $\rightarrow$ 3 values per pixel
- **Parameter scale:**
- First hidden layer with 1,000 units:
 - 3×10^9 weights (excluding bias parameters) - **Extremely large parameter count**
- Must learn invariances and equivariances from data alone – **require huge datasets**

Why Structure Matters



- Encode prior knowledge about image structure directly into the model:
 - **reduce parameter count**
 - **improve generalization with respect to symmetries in the image space**
 - **reduce data set requirements dramatically**

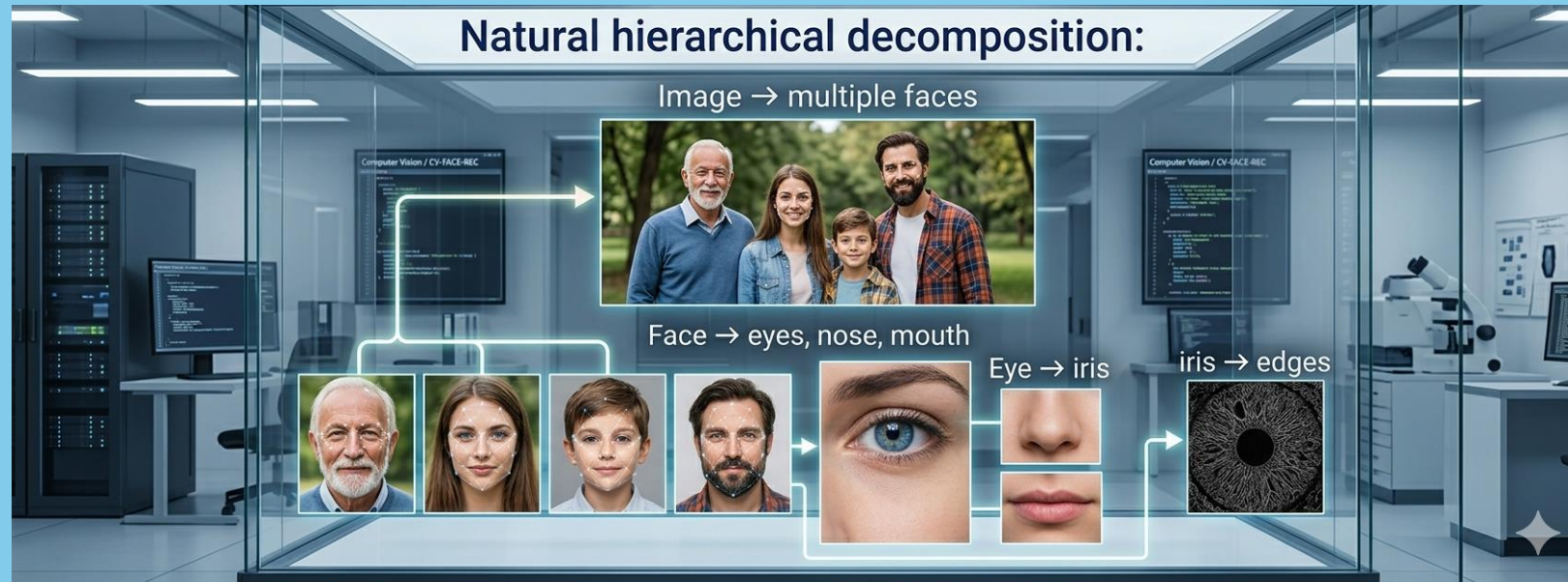
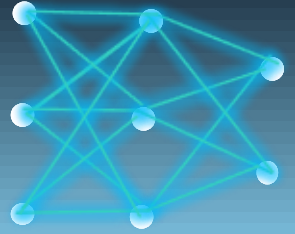
Four Key Inductive Biases in CNNs



- CNNs exploit four coupled concepts:
 - **Hierarchy**
 - **Locality**
 - **Equivariance**
 - **Invariance**

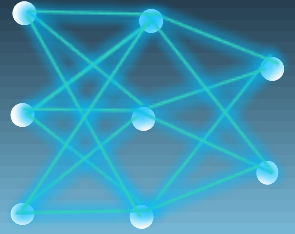
Hierarchical Structure in Vision

Example : face detection

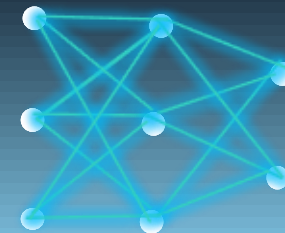


- Complex structures are composed from simpler ones

Locality: Building from Small Regions



- Early network units detect **local features**
- Example:
 - edges
 - corners
 - simple textures
- Each neuron would only need to see a small subset of the image pixels
- More complex structures further up the hierarchy can be detected by composing multiple features found at previous levels

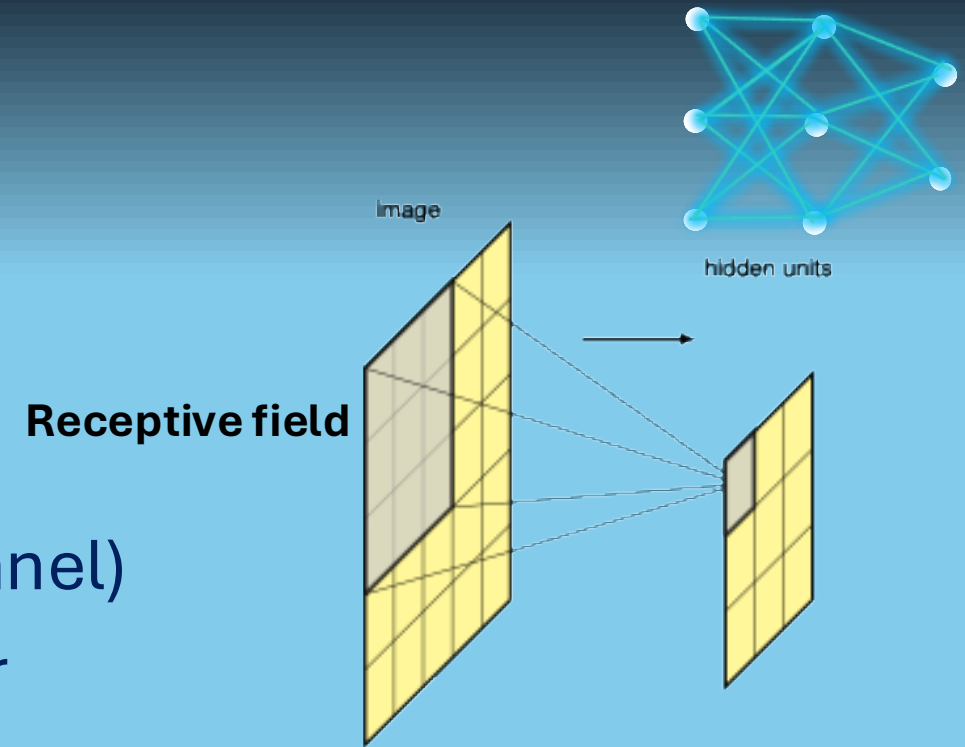


Equivariance and Invariance

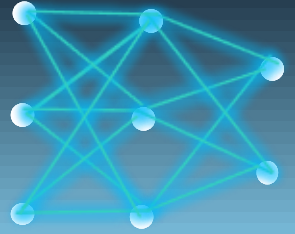
- CNNs aim to encode:
 - **Equivariance**: structured response to transformations
 - **Invariance**: robustness to irrelevant changes
- **Example**:
 - Object shift in image $\rightarrow$ feature map shifts similarly
 - Spatial transformations are handled systematically

Feature Detectors in CNNs

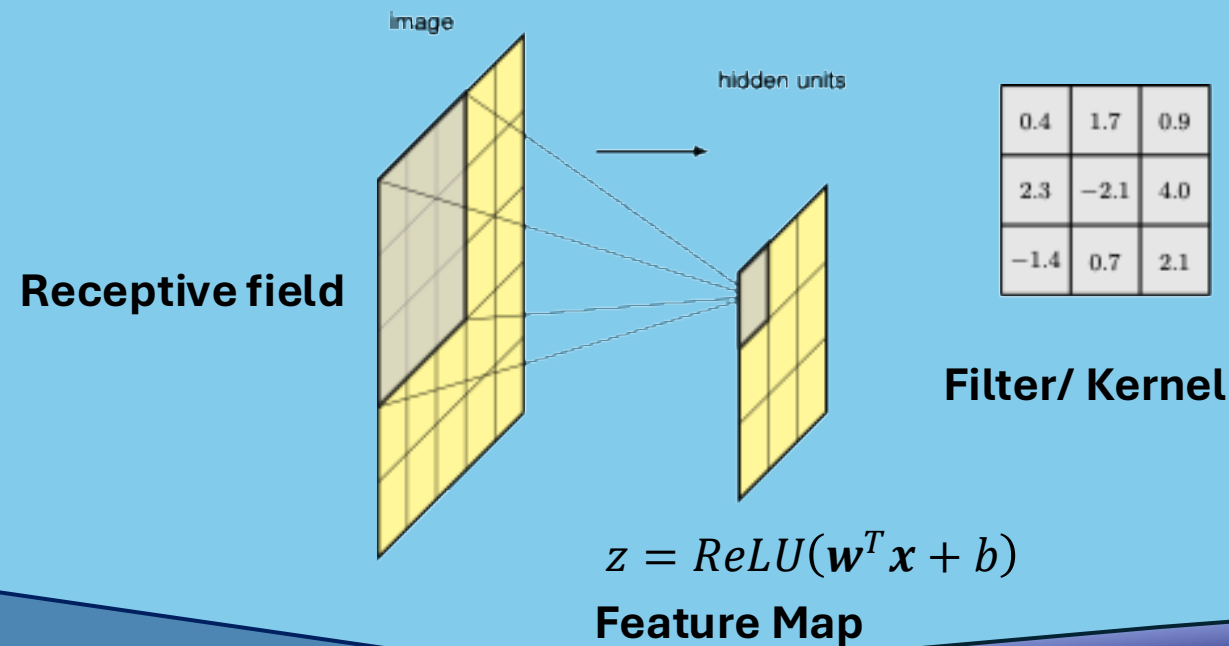
- Consider **grayscale images** (single channel)
- Focus on a single neuron in the first layer
- Neuron operates on a **small local image patch**
- $\Rightarrow$ Encodes locality: each unit sees only a region of the image



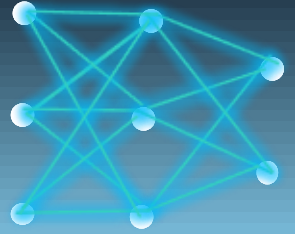
Local Linear Feature Computation



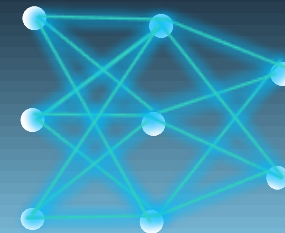
- Each neuron computes a local feature detector



Filters as Learned Patterns



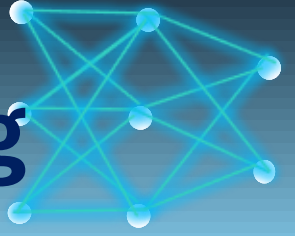
- Each input pixel has a corresponding weight
- Therefore:
 - w forms a **2D grid**
 - Can be visualized as an image pattern



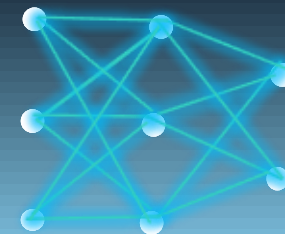
What Does a Filter Detect?

- For which input patch x is the neuron maximally activated?
- Maximum response occurs when **input matches the filter pattern**
- **Role of ReLU Activation:**
Acts as a **thresholded feature detector**
Neuron fires only for sufficiently good matches

Geometric Interpretation of Feature Matching



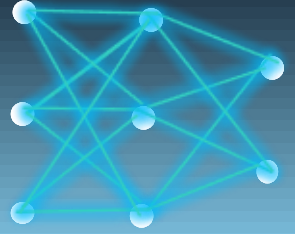
- Optimal input patch:
 - aligned with weight vector w
 - same structure up to scaling
- **Interpretation:**
 - Filter acts as a **template matcher**
 - Strong response when image patch $\approx$ learned pattern
- CNN neurons detect similarity to learned prototypes



Translation Equivariance: Core Motivation

- A local feature (e.g., an **eye**) should be recognized **anywhere in the image**
- Same pattern of pixels $\Rightarrow$ same semantic meaning, regardless of location
- **Desired property:** Learning at one location should generalize to all locations without needing to see examples in the training set of the corresponding feature at every possible location

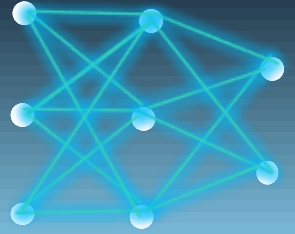
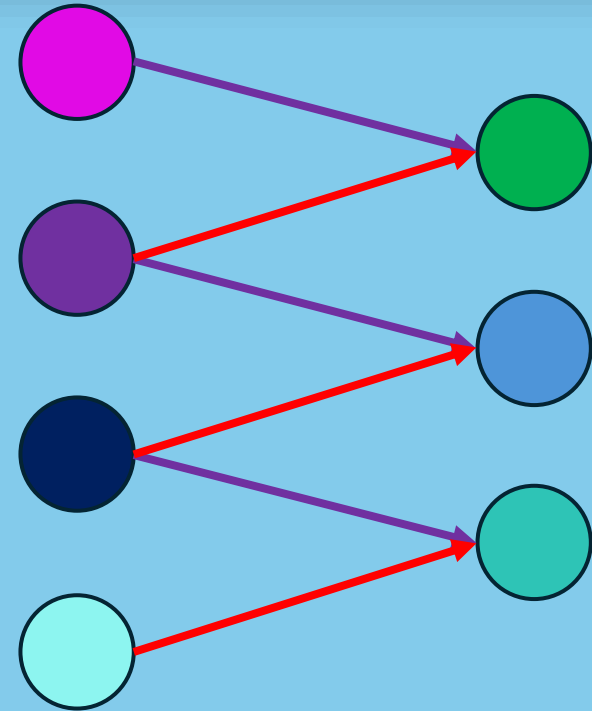
Weight Sharing and Feature Maps

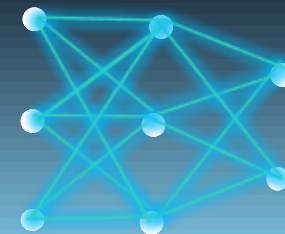


- Solution: **replicate the same weights across spatial positions**
- Same local patch $\rightarrow$ same response at any spatial location
- **Property:** The model is **translation equivariant**

Sparsity + Weight Sharing

- CNN structure introduces:
- **Sparse connectivity**
 - each neuron sees only a local patch
- **Weight sharing**
 - same filter applied across all locations



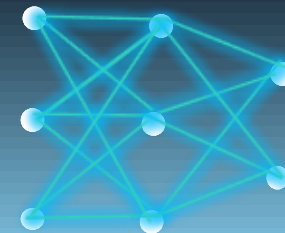


Mathematical Form of Convolution

- For an image I with pixel intensities $I(j, k)$ and a filter K with pixel values $K(l, m)$, the feature map $C = I * K$ (ignoring bias and nonlinear activation):

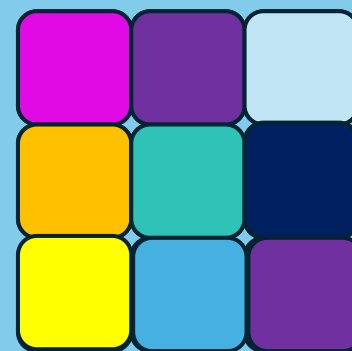
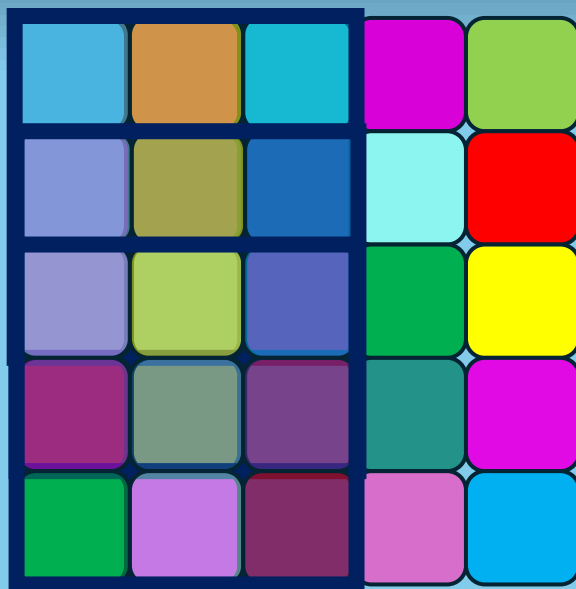
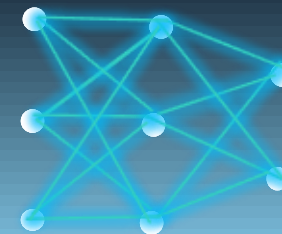
$$C(j, k) = \sum_l \sum_m I(j + l - 1, k + m - 1) K(l, m)$$

- Slide filter across image
- Compute local dot products
- Feature map = spatial correlation with learned kernel

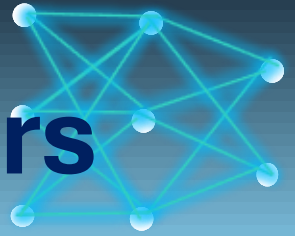


Convolution vs Cross-Correlation

- Technically **cross-correlation**
- Slight difference from classical convolution definition
- **Convention:**
 - ML literature refers to it as **convolution**



Example: Edge Detection with Fixed Filters



- **Vertical edge detector:** Detects horizontal intensity changes
- **Horizontal edge detector:** Detects vertical intensity changes

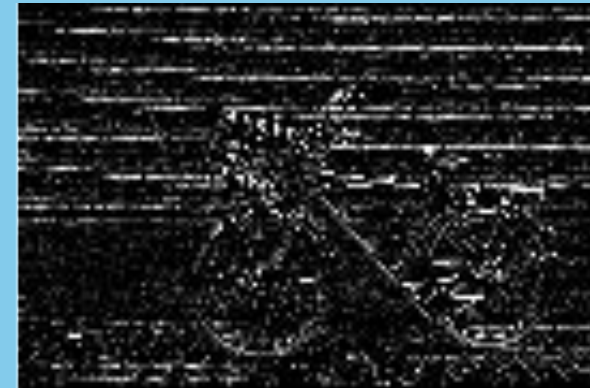
$$\begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}$$
$$\begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix}$$



(a)

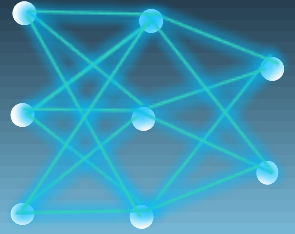


(b)



(c)

Advantages of Convolutional Networks



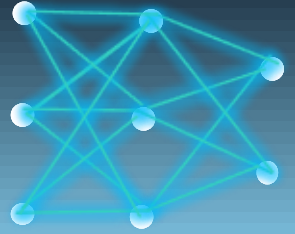
- Compared to fully connected networks:

Sparsity - far fewer connections

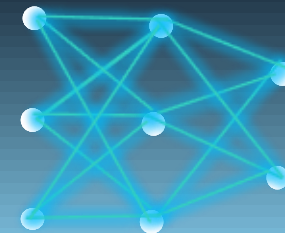
Weight sharing - drastically fewer parameters (consequently reducing the required size of the training set needed to learn those parameters)

Input size flexibility - same filters work for different image sizes

Computational Efficiency

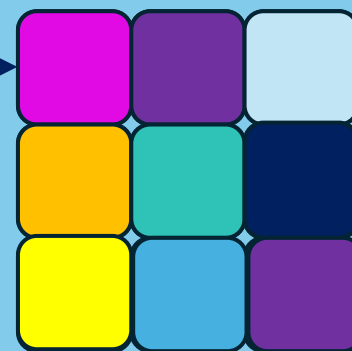
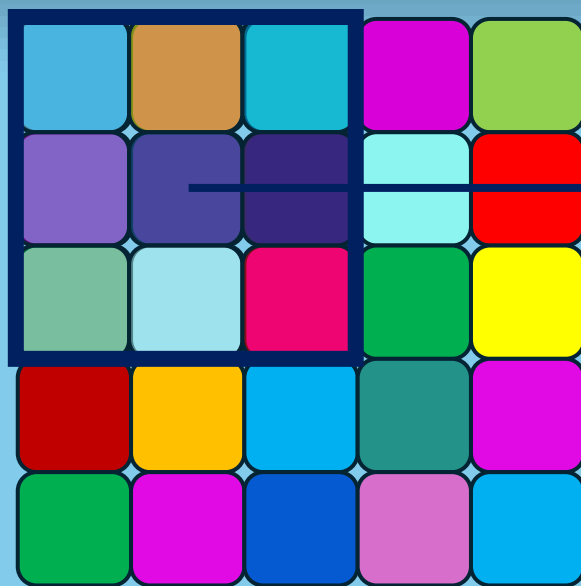
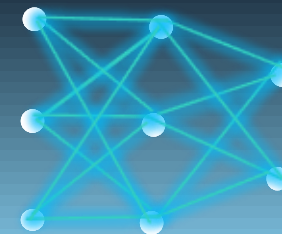


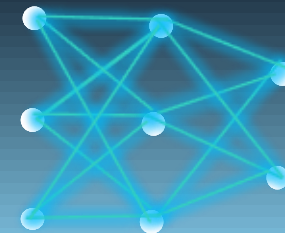
- Convolutions are highly parallelizable
- Efficient implementation on **GPUs**
- Scalable training on large images



Padding: Why Is It Needed?

- Standard convolution **reduces spatial dimensions**
- Input image dimensionality: $J \times K$
- Kernel dimensionality: $M \times M$
- Resulting feature map dimensionality:
$$(J - M + 1) \times (K - M + 1)$$
- Convolution shrinks feature maps



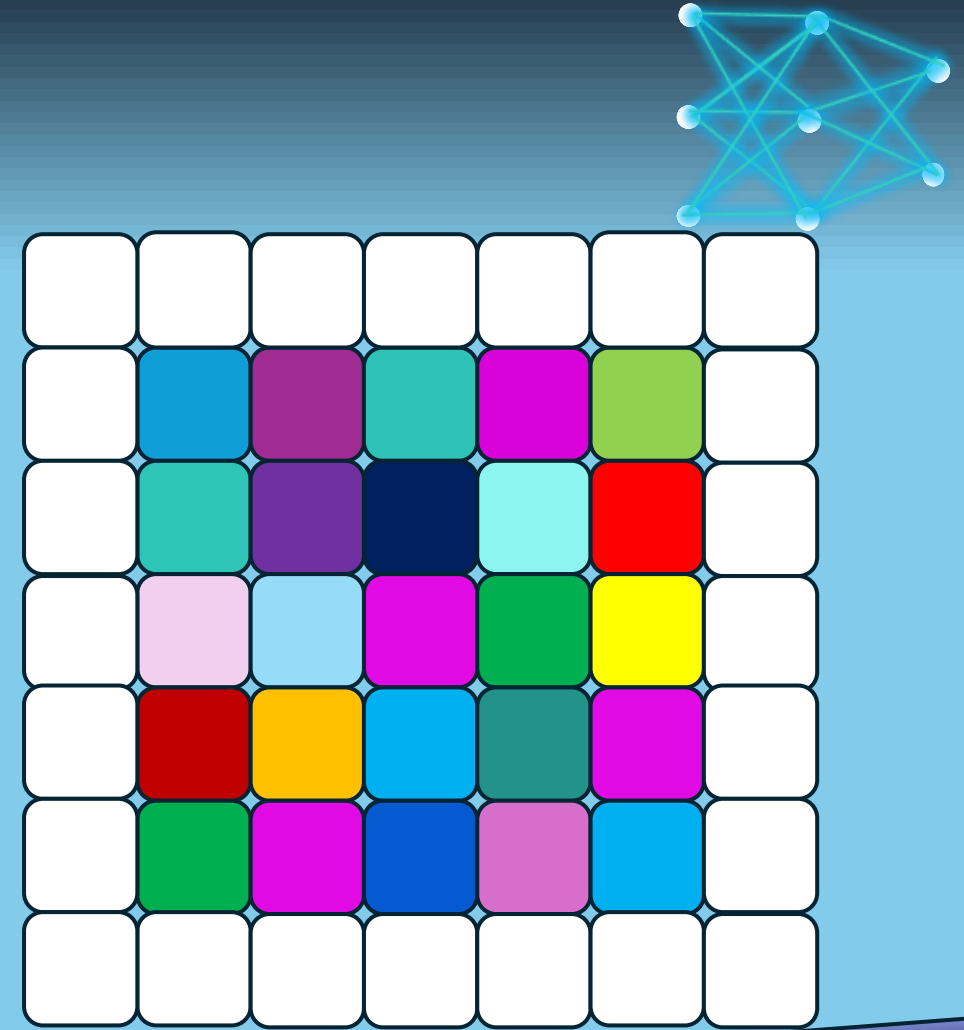


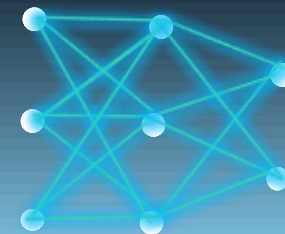
Problem with Size Reduction

- Repeated convolutions:
 - progressively reduce spatial resolution
- Loss of:
 - boundary information
 - spatial alignment with input
- Sometimes we want:
feature map dimension = original image dimension

Padding Solution

- Add extra pixels around image border
- Padding size: P
- **Output dimension:**
 $(J+2P-M+1) \times (K+2P-M+1)$





Types of Convolution

- **Valid Convolution**

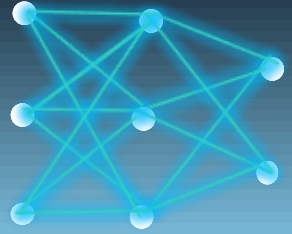
- ✓ No padding: $P = 0$
- ✓ Output shrinks

- **Same Convolution**

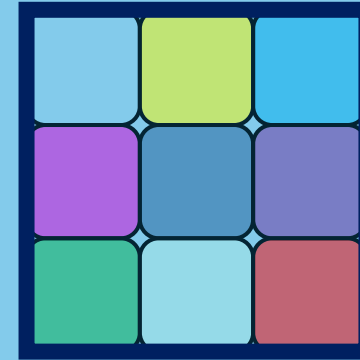
$$P = \frac{M - 1}{2}$$

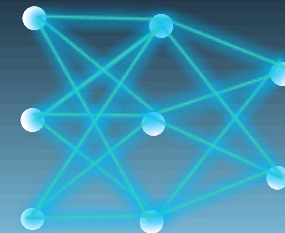
- ✓ The image and the feature map have the **same dimension**
- ✓ **Same convolution** preserves spatial dimensions

Why Odd-Sized Filters Matter



- Typically use odd M (e.g., 3×3 , 5×5)
- Allows symmetric padding
- Ensures a **center pixel exists**

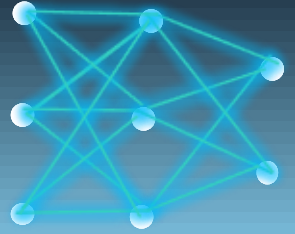




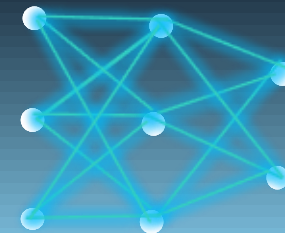
What Values to Use for Padding?

- Common choice: **zero padding**
- **Best practice:**
 - Subtract mean pixel intensity first
 - Then zero represents “average background”

Padding in Deep CNNs



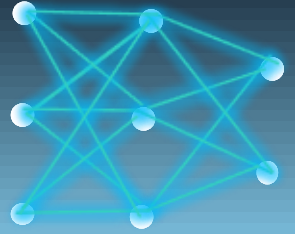
- Padding can be applied:
 - at input image level
 - between convolutional layers - preserves spatial resolution deeper in network



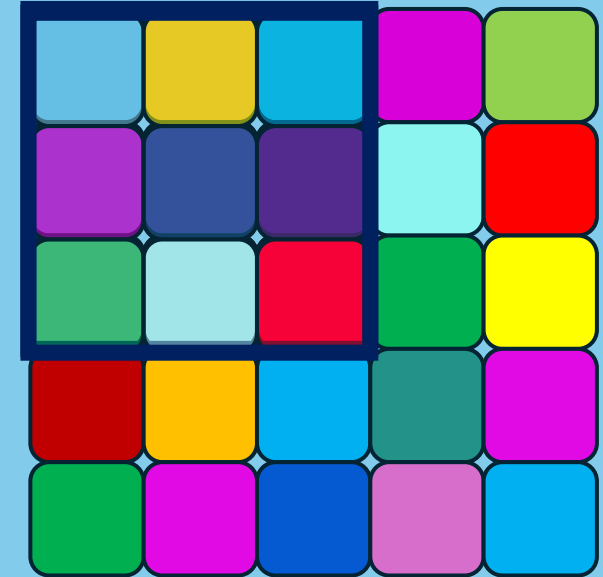
Strided Convolutions: Motivation

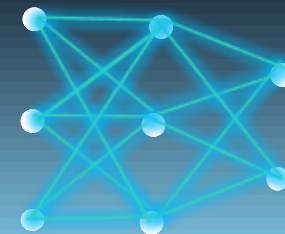
- **Images** can have **large number of of pixels**
- **Kernels** are often relatively **small** $M \ll J, K$
- Convolutional **feature maps** will be of **similar dimension** to the original image dimension (same for same convolution)
- Sometimes we wish to use **feature maps** that are **significantly smaller** than the original image to provide flexibility in the design of convolutional network architectures

Key Idea: Stride



- Instead of moving filter **one pixel at a time**
Move it in steps of size S
- **Stride S** = step size of filter movement
- Reduces spatial resolution of feature maps





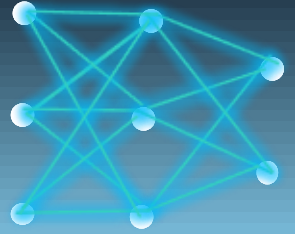
Effect on Feature Map Dimension

- For input dimension $J \times K$ and with the same stride horizontally and vertically, the number of elements in the feature map will be

$$\left\lfloor \frac{J+2P-M}{S} \right\rfloor + 1 \times \left\lfloor \frac{K+2P-M}{S} \right\rfloor + 1$$

- For large images and small filter dim, the image map will be roughly a factor of $1/S$ smaller than the original image

Effect on Feature Map Dimension



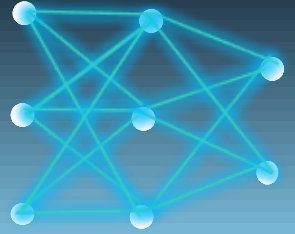
$$1, 1 + S, 1 + 2S, \dots, 1 + lS$$

$$1 + lS + (M - 1) \leq J + 2P$$

$$l \leq \frac{J + 2P - M}{S} \rightarrow l_{max} = \left\lfloor \frac{J + 2P - M}{S} \right\rfloor$$

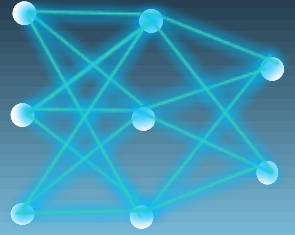
$$\left\lfloor \frac{J + 2P - M}{S} \right\rfloor + 1 \times \left\lfloor \frac{K + 2P - M}{S} \right\rfloor + 1$$

Multi-Dimensional Convolutions: Motivation

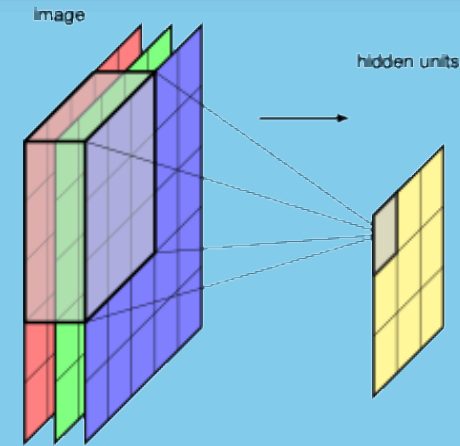


- Real images are **multi-channel (RGB)**
- **Representation:** Image tensor $J \times K \times C$
- where: J, K - spatial dimensions
 C : number of channels (e.g., 3 for **RGB**)
- Convolution must operate across spatial + channel dimensions

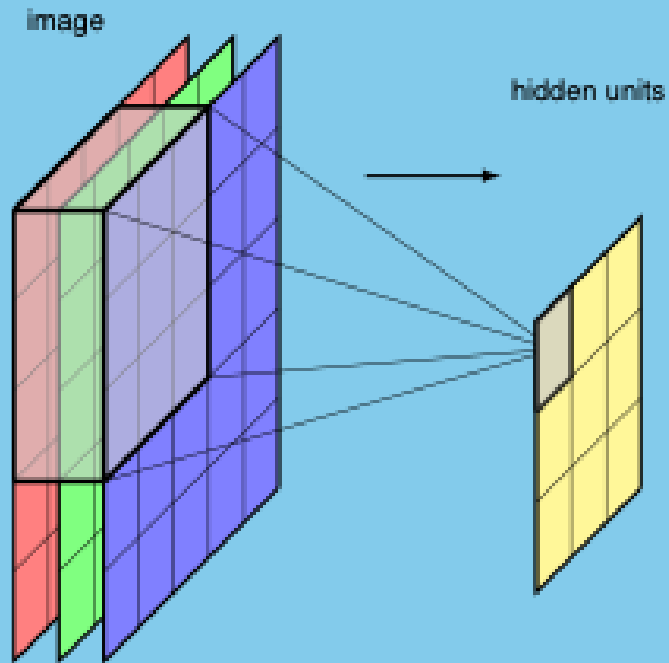
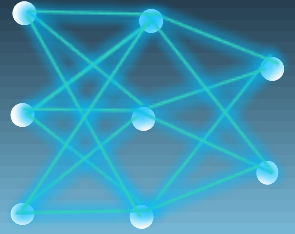
Filters for Multi-Channel Images



- A filter must span **all input channels**
- **Filter shape:** $M \times M \times C$
One $M \times M$ kernel per channel
- Outputs a **single feature map**
- **Output size (stride 1, no padding):** $(J - M + 1) \times (K - M + 1)$
- Each filter combines spatial + cross-channel information

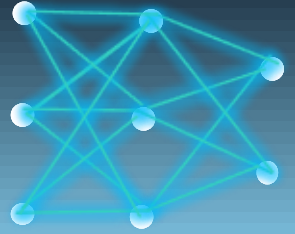


Filters for Multi-Channel Images

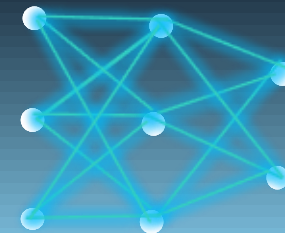


1.2	0.8	3.7	
-3.2	0.7	1.3	
-3.1	0.4	1.7	0.9
2.1	2.3	-2.1	4.0
4.1	-1.0	0.7	2.1

Parameter Count of a Single Filter



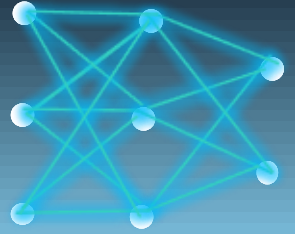
- For one filter: $M^2C + 1$ parameters (weights + bias)
- Independent of image dimension J, K
- Parameter efficiency remains despite high-dimensional inputs



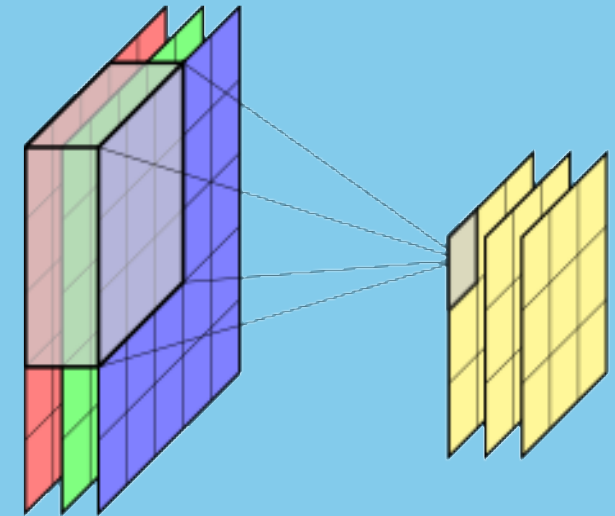
Limitation of a Single Filter

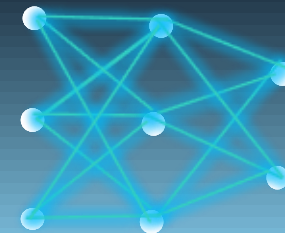
- One filter produces **one feature map**
- Equivalent to one hidden neuron in a fully connected layer
- **Limitation:** Can only detect **one type of feature**
- **Examples:**
 - one edge type
 - one texture pattern

Multiple Filters and Feature Maps



- Use multiple filters: each with independent parameters
- Each filter produces its own feature map
- Multiple filters = richer feature representation





Full Tensor Formulation

- The filter tensor now has dimensionality

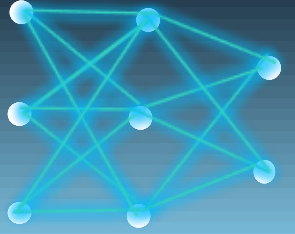
$$M \times M \times C \times C_{out}$$

C is the number of input channels

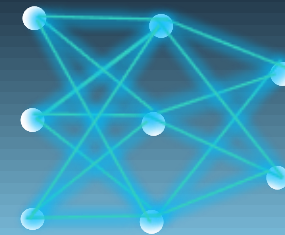
C_{out} is the number of output channels

- **Each output channel** has its own bias
- **Total parameters:** $(M^2C + 1)C_{out}$
- Expressive capacity scales with number of filters

Interpretation of Channels



- Input channels:
 - RGB or learned feature representations
- Output channels:
 - learned feature detectors



Thank you